Efficient Range Deletes in LSM Engines

Yu-Cheng Huang
Boston University
USA

****
Boston University
USA

****
Boston University
USA

ABSTRACT

In contemporary times, the LSM-Tree is widely adopted for its efficient insertion capabilities. However, a drawback arises when dealing with range deletes, as they are stored in individual SST files. Consequently, the process of searching for a deleted key necessitates opening and reading a file, leading to wasteful disk IO operations and time consumption.
This paper focuses on addressing this issue by proposing a range delete filter (RDF) to minimize disk IOs during the search for deleted keys. The RDF serves as a mechanism to swiftly identify and handle deleted keys, optimizing the overall performance of the LSM-Tree.

ACM Reference Format:
Yu-Cheng Huang, ****, and ****. 2023. Efficient Range Deletes in LSM Engines. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

LSM-based Key-Value Stores. With the arrival and development of 5G, AGI, XR, more and more data are generated and stored on the servers. A bunch of database system adopt LSM as its underlying storing structure for its writing efficiency. Data first write directly into buffer until the buffer is full, then the whole data block is flushed on to the disk as an immutable SSTable with keys sorted. There're several levels in which their size grows larger with ratio T. Once the level reaches its maximum capacity, several SSTable files are gathered and merged-sorted before moving to the next level.
Range Delete in LSM Trees. Typically, range deletes are stored as an range delete tombstones with start and end keys indicating their coverage inside the SSTable files where the file ranges overlapped with the key ranges. –¿ In RocksDB, when compaction happens, these range deletes purge out the obsolete entries across all the compacted files.
Problems. We may waste time searching for deleted items on the disk because range deletes are stored in the SSTable files. To know whether an entry exist, there's a chance we have to open and read the file if not blocked by fencing index and bloom filter. However, there is a huge speed gap between fetching data from disk and from the memory, which ends up slowing our queries. (–¿Assumption)

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1.1 Motivation

If we can use range delete information to set an filter (RDF) inside the buffer to decrease numbers of IO access, then the speed of data query can be accelerated. (–¿ Assumption) With RDF, we can directly purge out queries on non-existed keys and prevent unnecessary disk IOs.

1.2 Problem Statement

Ideally no files shall be read and can directly return a non-existence result when the searching keys have been removed by range deletes beforehand because all the keys fall inside the deleted range don't exist anymore. In this case, there shall be no disk IO but range deletes are stored inside

Range deletes possess information whether an entry exist because all the entry inserted earlier are deleted. Currently, range deletes are directly stored into SST file as what Rocksdb does. However, with range deletes information, we can directly reject PQ for reaching deeper level searching on entries that are inserted ealier than range deletes even when bloom filter and

1.3 Contributions

(a) discuss and analyze influence of different RDFs (split-PLRDF, toplevel-RDF) and their memory footprint
(b) get rid of the requirement of sequence number for every entry

2 LSM BACKGROUND

The LSM tree is a structure optimized for writes and consists primarily of two operations: Flush and Compaction. It consists of multiple levels with T-ratio up-growing capacity.
Leveling and Segmented LSM. Data is stored in SSTable files on each level, with non-overlapping ranges between files. During compaction, only selected files are moved to the next level.
Flush. When the buffer is full, data is sorted and written into an immutable sorted string table on disk.
Compaction. Triggered when a level is full, several files will be picked out following some rules and sort-merged with the overlapping files on the next level.
Write Amplification and Space Amplification. Space amplification occurs due to the presence of the same key on different levels. Write amplification happens when duplicate keys in the database result in additional writes when moving

data to the next level.

Range Delete. Range deletes are stored as range tombstones, represented by a triple of start value, end value, and sequence number in each SSTable file that contains their ranges.

3 PROBLEM

Range deletes contain data about non-existent entries but can only be retrieved from files, leading to additional disk IOs and a reduction in query speed. If we had a filter capable of determining whether ranges are deleted, we could minimize unnecessary disk IOs. Currently, the bloom filter stores only point entry information, and fencing pointers store existing file ranges, but no filter is storing delete ranges, telling whether a point is deleted.

4 SOLUTION NAME

4.1 Solution Design

RD can be either stored on each level (PLRDF) or on the top level (TopLevel-RDF). When range deletes flush down to the disk, range deletes are merged and stored on to level 0 vector. Latter, when compaction happens, corresponding range deletes are moved and merged to the next level vector in PLRDF. While in TopLevel-RDF, range deletes are stored in the top level only, and those whose ranges have intersected with the newly inserted files are removed. When either flush or compaction happens, range deletes from the topper level which are newer remove the obsolete entries from the files it compacted to on the lower level. In this way, if an entry is overlapped with a range delete tombstone in a file, it must be newer than the range delete tombstone. Sequence numbers are not required because if there are overlapping files across two levels, files on the lower level is newer and if an entry and range deletes co-exist in a file, the entry is sure to exist.

(a) PLRDF: Range deletes are stored on each level and moved down to the next level during compaction. When a point query is blocked on a certain level, it indicates that the key doesn't exist on the levels below, requiring disk access on the current level, just the same as storing range deletes inside SST files..
(b) Split-PLRDF: Whenever new entries arrive at a certain level, ranges that overlap with the newly inserted entries are split. This ensures that an entry existing on a certain level won't be covered by ranges in the RDF. However, this approach results in a larger memory footprint due to fragmented ranges.
(c) TopLevel-RDF: RDF is limited to the top level, such that it blocks query if an entry doesn't exist in the database. It performs split function on all incoming entries, leading to larger memory footprint.

5 EVALUATION

6 RELATED WORK

7 CONCLUSION

REFERENCES